

ForgeOS

End-to-End Orchestration Document

Demand-to-Product Pipeline: Complete Walkthrough

From Client Demand to Delivered Product
Covering all roles, systems, and decision gates

ForgeOS Demand Delivery Pipeline v1.0

ForgeOS - End-to-End Orchestration Document

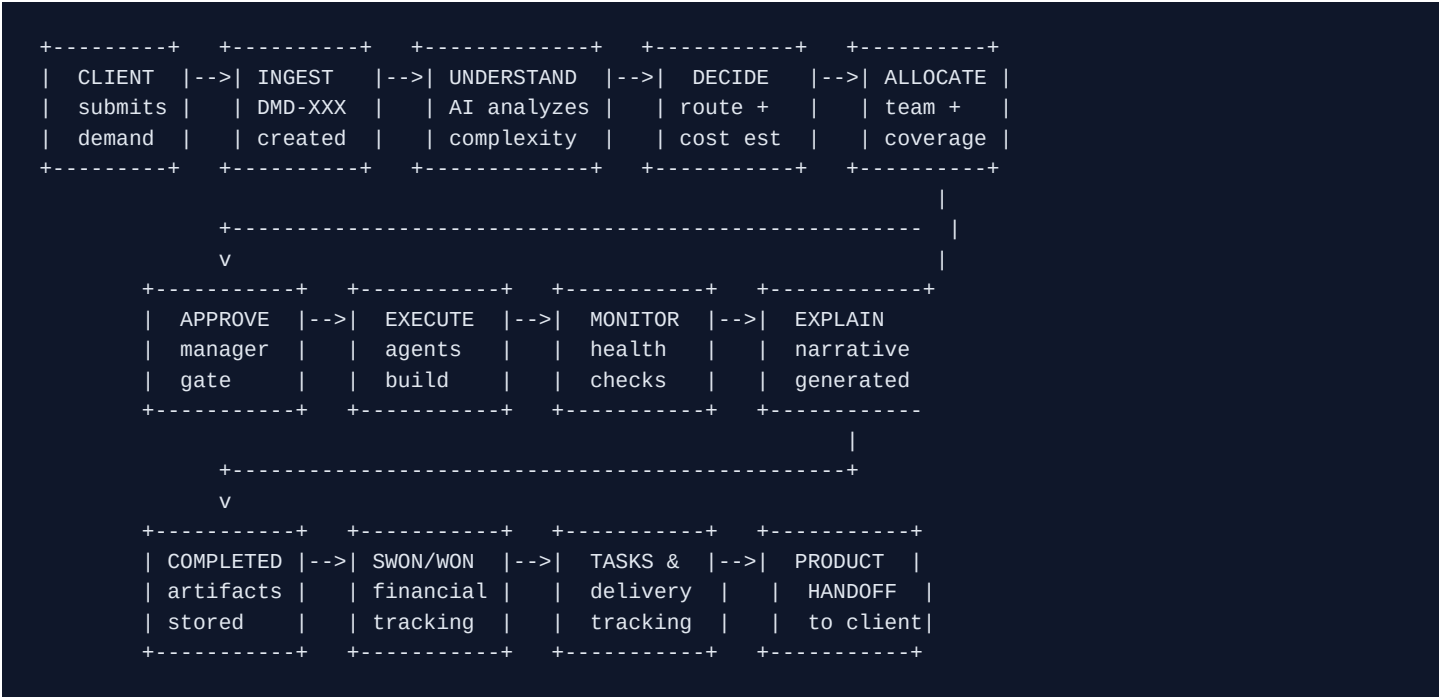
Demand-to-Product Pipeline: Complete Walkthrough

*This document traces a single demand from the moment a **client** voices a need, through AI planning, manager approval, autonomous execution, delivery tracking, and final product handoff. Every role, system component, API call, and decision gate is covered in order.*

Table of Contents

- * [Pipeline Overview](#1-pipeline-overview)
- * [Stage Lifecycle](#2-stage-lifecycle)
- * [Step 1 - Client Submits a Demand](#3-step-1--client-submits-a-demand)
- * [Step 2 - Ingestion](#4-step-2--ingestion)
- * [Step 3 - AI Understanding](#5-step-3--ai-understanding)
- * [Step 4 - Reuse Detection](#6-step-4--reuse-detection)
- * [Step 5 - Execution Decision](#7-step-5--execution-decision)
- * [Step 6 - Resource Allocation](#8-step-6--resource-allocation)
- * [Step 7 - Manager Plan Review](#9-step-7--manager-plan-review)
- * [Step 8 - Approval Gate](#10-step-8--approval-gate)
- * [Step 9 - Background Execution](#11-step-9--background-execution)
- * [Step 10 - Monitoring](#12-step-10--monitoring)
- * [Step 11 - Explanation Generation](#13-step-11--explanation-generation)
- * [Step 12 - Completion & Artifacts](#14-step-12--completion--artifacts)
- * [Step 13 - Delivery Tracking (SWON / WON / Tasks)](#15-step-13--delivery-tracking-swon--won--tasks)
- * [Step 14 - Dashboards & Reporting](#16-step-14--dashboards--reporting)
- * [Step 15 - Product Handoff to Client](#17-step-15--product-handoff-to-client)
- * [Cross-Cutting Concerns](#18-cross-cutting-concerns)
- * [Role Involvement Matrix](#19-role-involvement-matrix)
- * [Worked Example - Insurance Claims Portal](#20-worked-example--insurance-claims-portal)

1. Pipeline Overview



Duration: Minutes (AI-agent mode) to weeks (human-team mode).

Actors: Client -> System (AI) -> Manager -> Middleware -> Leader -> Delivery Team -> Higher Manager -> Executive.

2. Stage Lifecycle

#	Stage	Owner	Duration	What Happens
1	`ingested`	System	< 1 second	Demand ID assigned, raw text stored
2	`understanding`	AI Engine	2-10 seconds	NLP analysis of complexity, domain
3	`deciding`	AI Engine	2-10 seconds	Route decision + cost/time estimate
4	`allocating`	AI Engine	1-5 seconds	Team formation + coverage scoring
5	`awaiting_approval`	Manager	Minutes-days	Human review gate
6	`executing`	AI Agents	1-30 minutes	Code generation + artifact creation
7	`monitoring`	System	< 1 minute	Health checks on agent runs
8	`explaining`	AI Engine	2-5 seconds	Human-readable delivery narrative
9	`completed`	System	Instant	Artifacts stored, demand closed
-	`failed`	System	-	Error recovery path
-	`cancelled`	Manager	-	Manual cancellation

Defined in: backend/app/schemas.py -> DemandStage enum.

Transition logic: backend/app/planner/pipeline.py -> PlannerPipeline.stage_after().

3. Step 1 - Client Submits a Demand

Who: Client (external) or Internal User (manager/member)

There are two intake paths, both leading to the same pipeline.

Path A: Internal User - 3-Step Wizard

Frontend: frontend/src/routes/demand/new.tsx

The user fills out a guided form:

Field	Options
Industry	14 choices (Banking, Insurance, Healthcare, Retail
Priority	Low, Medium, High
Timeline	< 1 month, 1-3 months, 3-6 months, > 6 months
Budget Range	< INR 10L, INR 10L-50L, INR 50L-2Cr, > INR 2Cr
Requirement	Free-text description of the project

Quick-start templates are available for common project types.

The system concatenates all metadata into a structured text block:

```
Industry: Insurance
Priority: High
Timeline: 1-3 months
Budget range: INR 50L-2Cr
Requirement: Build a multi-channel motor insurance claims portal
with AI-powered damage assessment from uploaded photos...
```

API Call: POST /api/demands with { text: enrichedText }.

Path B: External Client - Simplified Submission

Frontend: frontend/src/routes/client/index.tsx

The client provides:

- * Contact name and email
- * Free-text project description
- * The system auto-detects industry/priority from the text

API Call: POST /api/portal/requests with { client, description }.

This path creates a PortalRequest record linked to the demand, plus initial PortalMessage entries (client message + system acknowledgment). The demand proceeds through the same AI pipeline but is flagged for mandatory manager review before launch.

What the Client Sees After Submission

- * A confirmation banner with the demand ID (e.g., DMD-3A7F2B1C90)
- * A status tracker showing the demand is "Processing"
- * The client can check status via the client portal at any time

4. Step 2 - Ingestion

Who: System (automatic)

Stage: `ingested`

Backend: backend/app/planner/ingestion.py -> DemandIngestion.ingest()

Action	Detail
Generate public ID	`DMD-{uuid_hex[:10].upper()}` (e.g., DMD-3A7F2B)
Wrap into `DemandInput` schema	Captures text, source, char/word count
Create DB record	`DemandRequest` row with `stage = "ingested"`
Audit event	`action = "created"` logged automatically

Output: A persisted demand record with a unique public ID, ready for AI analysis.

Transition: Stage immediately advances to understanding.

5. Step 3 - AI Understanding

Who: AI Engine (LLM-powered)

Stage: `understanding`

Backend: backend/app/planner/understanding.py -> UnderstandingEngine.analyze()

The system analyzes the raw demand text to extract structured intelligence:

Output Field	Example Value
`problem_type`	`web_application`
`domain`	`insurance`
`complexity`	`HIGH`
`urgency`	`HIGH`
`required_skills`	`["react", "python", "computer_vision", "postgresq
`key_features`	`["claims portal", "damage assessment", "fraud che
`estimated_scope_days`	`45`
`summary`	"Multi-channel insurance claims management..."

LLM Flow

- * Primary path: Resolve model via `model_router.resolve("understanding")`, send `UNDERSTANDING_PROMPT` with demand text, parse JSON response.
- * Fallback path: If LLM fails, a deterministic `_heuristic()` function performs keyword matching to classify the demand.

LLM Provider Chain

All LLM calls route through a FallbackProvider with automatic cascading:

```
Primary (NIM/vLLM/Groq/OpenAI) -> Tier 1 (Groq) -> Tier 2 (OpenRouter) -> Browser Bridge (WebLLM)
```

Retryable errors (429, 5xx, timeouts) cascade down the chain. 2 attempts per tier with jittered retry.

Transition: Stage advances to deciding.

6. Step 4 - Reuse Detection

Who: AI Engine (vector search)

Runs during: `understanding` -> `deciding` transition

Backend: backend/app/db/vector.py -> ReuseDetector.find_similar()

This step searches the organization's past project library for components that can be reused, saving time and cost.

How It Works

- * Embed the demand text using the configured embedding model.
- * HNSW kNN search over past_projects.embedding using pgvector cosine distance.
- * Return the top 3 matches with similarity scores.
- * Fallback: If embedding fails, use Jaccard word-overlap similarity with domain/type bonuses.

Example Output

Past Project	Similarity	Reuse Rationale
Healthcare Patient Portal	0.82	Auth module, role-based dashboards, document upload
Retail Inventory Management System	0.71	Real-time tracking patterns, notification framework
Banking Loan Processing Platform	0.68	Approval workflows, compliance audit trail

Impact on Downstream Decisions

Reuse Score	Effect on Decision
> 0.7	System recommends `REUSE_EXISTING` mode - adapt, d
0.4 - 0.7	Components flagged as candidates; team reviews dur
< 0.4	Fresh build recommended; reuse opportunities minim

7. Step 5 - Execution Decision

Who: AI Engine (LLM or rule-based)

Stage: `deciding`

Backend: backend/app/planner/decision.py -> DecisionEngine.decide()

The system determines how the demand should be fulfilled.

Four Execution Modes

Mode	When Chosen	What It Means
`AI_AGENT`	Low complexity, or web apps/chatbots	Fully autonomous AI agents build the product
`HYBRID`	Medium complexity, or high-urgency + human_	AI agents + human oversight working together
`HUMAN_TEAM`	High complexity	Traditional human delivery team, AI-assisted
`REUSE_EXISTING`	Reuse score > 0.7	Fork and adapt existing components

Decision Output

Field	Example Value
`execution_mode`	`HYBRID`
`project_type`	`web_application`
`reasoning`	"Medium complexity with strong reuse candidates..."
`estimated_cost_usd`	`12,500`
`estimated_time_days`	`30`
`confidence_score`	`0.78`
`risk_factors`	`["integration complexity", "third-party API depen
`reuse_percentage`	`35`

Transition: Stage advances to allocating.

8. Step 6 - Resource Allocation

Who: AI Engine

Stage: `allocating`

Backend: backend/app/planner/allocation.py -> AllocationEngine.allocate()

The system builds an optimal team from a 50-person resource pool (33 humans + 13 AI agents + 4 partner vendors).

Team-Building Strategies

Execution Mode	Strategy	Team Composition
`AI_AGENT`	`_ai_team()`	5-7 AI agents + 2 human reviewers (non-low)
`HUMAN_TEAM`	`_human_team()`	8-11 humans + partner vendors for high complexity
`HYBRID`	`_hybrid_team()`	5-6 AI agents + 4-6 humans
`REUSE_EXISTING`	`_reuse_team()`	5 best-fit resources from entire pool

Selection Algorithm

- * Skill expansion - Derives required skills from understanding output + problem type + features + complexity.
- * Ranking - Each candidate scored by `_rank()` (skill-match percentage + seniority weighting).
- * Coverage - Greedy set-cover via `_cover()` to maximize skill coverage with minimum team size.

Allocation Output

Field	Example Value
`team`	9 members (names, roles, skills, daily cost)
`total_daily_cost`	`INR 45,000`
`coverage_score`	`0.92`
`uncovered_skills`	`["computer_vision"]`
`allocation_reasoning`	"Hybrid team with frontend focus..."
`bench_size`	`3` (standby resources)

What the Plan Looks Like at This Point

The demand now has a complete plan:

- * Understanding:** What the project is, its complexity, required skills
- * Decision:** How to build it, cost/time estimates, risk factors
- * Allocation:** Who will build it, skill coverage, daily cost
- * Reuse Matches:** Which past projects can be leveraged, and why

Transition: Stage advances to `awaiting_approval`. The plan is returned to the frontend for human review.

9. Step 7 - Manager Plan Review

Who: Manager (human)

Stage: ``awaiting_approval``

Frontend: Step 2 of the demand wizard (`frontend/src/routes/demand/new.tsx`)

The manager reviews the AI-generated plan, which displays:

Plan Review Screen

```

+-----+
| PLAN: DMD-3A7F2B1C90 |
| |
| +- Understanding -----+
| | Domain: Insurance | Complexity: HIGH | Urgency: HIGH |
| | Skills: React, Python, CV, PostgreSQL, Redis |
| | Scope: ~45 days | Features: 6 key components |
| +-----+
| |
| +- Decision -----+
| | Mode: HYBRID | Cost: $12,500 | Time: 30 days |
| | Confidence: 78% | Reuse: 35% |
| | Risks: Integration complexity, 3rd-party API dep |
| +-----+
| |
| +- Allocation -----+
| | Team: 9 members | Coverage: 92% |
| | Daily Cost: INR 45,000 | Bench: 3 standby |
| +-----+
| |
| +- Reuse Matches -----+
| | 1. Healthcare Portal (82%) - Auth, dashboards, upload |
| | 2. Retail Inventory (71%) - Tracking, notifications |
| | 3. Banking Loan (68%) - Workflows, audit trail |
| +-----+
| |
| [ Consult AI Copilot ] [ Approve & Launch ]
+-----+

```

Manager AI Copilot

Before approving, the manager can consult an AI copilot for guidance:

API: POST /api/demands/{public_id}/manager-chat

The copilot has full context (demand + plan) and is system-prompted to advise on:

- * Scope assessment and risk analysis
- * Team composition adequacy
- * Approval readiness checklist
- * Client-facing questions to clarify
- * Cost-benefit tradeoffs

Manager Actions

Action	Result
Approve	Demand moves to `executing`, background job starts
Reject	Demand moves to `cancelled` (with reason in audit)
Reassign	Demand transferred to another manager/leader
Stage Change	Manually move to any valid stage (with reason)

10. Step 8 - Approval Gate

Who: Manager -> System

Transition: `awaiting\_approval` -> `executing`

API: POST /api/demands/{public_id}/approve

Backend: backend/app/api/demand.py -> approve_demand()

Gate Logic

- * Validate demand exists and is in awaiting_approval or failed state (retry allowed).
- * Transition demand to executing stage.
- * Enqueue background pipeline via enqueue_pipeline() -> Arq job on Redis.
- * Return confirmation to frontend.

High-Risk Safety Gate (Frontend)

For demands flagged as high-risk (execution mode is human_team OR 3+ risk factors), the frontend requires:

- * The manager to type "launch" as confirmation
- * This is enforced by the Gate component in Step 3 of the wizard

Audit Trail

The approval event is automatically captured:

- * entity_kind: "demand", action: "stage_changed", diff: { from: "awaiting_approval", to: "executing" }

11. Step 9 - Background Execution

Who: AI Agent Orchestra

Stage: `executing`

Backend: backend/app/queue/worker.py -> run_full_pipeline()

Executor: backend/app/executor/orchestrator.py -> Orchestrator.execute_project()

This is the core autonomous building phase. The system runs as an Arq background job with a 1-hour timeout.

Step 9.1 - Re-run Planner Stages (with real-time events)

Even though planning already ran during creation, the worker re-executes it to:

- * Get fresh data objects for the executor
- * Emit real-time WebSocket events so the frontend can show live progress

Events emitted: pipeline.understanding, pipeline.reuse, pipeline.decision, pipeline.allocation.


```
+-----+ +-----+
| Frontend Dev | | Backend Dev |
| Photo Upload | | Damage API   |
| (needs Form) | | (needs Auth) |
+-----+ +-----+
```

The Six Executor Agents

Agent	ID	Produces
Project Manager	`project_manager`	Task decomposition plan (6-10 tasks)
Frontend Developer	`frontend_dev`	React + TypeScript + Tailwind components
Backend Developer	`backend_dev`	Supabase SQL migrations + API logic
DevOps Engineer	`devops`	Dockerfile, docker-compose, .env.example
QA Tester	`qa_tester`	Test files + code review report
Documentation	`documentation`	README.md, SETUP.md

Each agent:

- * Receives its task + any dependency context
- * Streams code via LLM, emitting agent.code events in real-time
- * Parses output files from markdown code blocks
- * Returns { files: [{path, content}], raw }

Step 9.5 - Automatic QA & Documentation

If the PM agent's plan didn't include QA or documentation tasks, the system automatically runs both agents at the end to ensure:

- * Every project has tests
- * Every project has documentation

Step 9.6 - Post-Processing Safety Nets

Function: Orchestrator._post_process()

Three automatic fixes to ensure the generated code actually works:

Check	Fix
`App.tsx` has no real content	Rewrite to import + render all generated component
`index.html` broken	Fix ` <script>` entry point, inject Tailwind CDN</td></tr> <tr> <td>Supabase client overwritten</td><td>Restore the template Supabase client library</td></tr> </table> </div> <div data-bbox="47 772 300 791" data-label="Section-Header"> <h2>Step 9.7 - Artifact Storage</h2> </div> <div data-bbox="67 801 875 883" data-label="List-Group"> * Walk the project directory, collect all generated files (skip node_modules, .vite, dist). * Upload entire directory to artifact store (S3 or local filesystem) at tenants/{tenant_id}/projects/{project_id}. * Persist Artifact records in DB (path, size, SHA256 hash). * Persist AgentRun records for each agent's execution. </div> <div data-bbox="452 955 535 970" data-label="Page-Footer"> <p>Page 12 / 23</p> </div></script>

Real-Time Events During Execution

The frontend receives live WebSocket updates throughout:

Event	When	What's Shown
`pipeline.stage`	Each stage transition	Progress bar advances
`agent.status`	Agent starts/finishes	Agent card status changes
`agent.code`	During code generation	Live code streaming preview
`agent.log`	Agent log messages	Activity feed
`project.completed`	All agents done	Success notification

Events flow: Worker -> Redis Pub/Sub -> WebSocket Gateway -> Browser.

12. Step 10 - Monitoring

Who: System (automatic)

Stage: `monitoring`

Backend: backend/app/planner/monitoring.py -> MonitoringEngine.check()

After execution completes, the monitoring engine inspects all agent runs for issues:

Issue Type	Detection	Corrective Action
Task delays	Agent status is `"delayed"`	`reassign_support` - add helper
Slow progress	In-progress 4+ min but below 60%	`boost` - spin up DevOps in parallel
Overload	Same agent has 3+ concurrent tasks	`load_balance` - redistribute

If no issues: immediate transition to explaining.

13. Step 11 - Explanation Generation

Who: AI Engine

Stage: `explaining`

Backend: backend/app/planner/explanation.py -> ExplanationEngine.generate()

The system generates a human-readable narrative summarizing the entire delivery:

"ForgeOS delivered your Multi-Channel Motor Insurance Claims Portal using a hybrid execution approach. 9 team members (5 AI agents + 4 human specialists)

collaborated over 30 days, generating 47 files across frontend, backend, and infrastructure. The project achieved 92% skill coverage with an estimated cost of \$12,500. Key reusable components from the Healthcare Patient Portal were adapted for the authentication and dashboard modules, saving approximately 35% development time."

This explanation is stored on the demand record and shown to the client.

14. Step 12 - Completion & Artifacts

Who: System

Stage: `completed`

Backend: backend/app/queue/worker.py (final section)

Action	Detail
Set `completed_at`	Timestamp recorded on the demand
Store explanation	Narrative text persisted
Emit `pipeline.completed`	Final WebSocket event with file list
All artifacts accessible	Files available via artifact store API

What's Now Available

```
tenants/{tenant_id}/projects/DMD-3A7F2B1C90/
+-- src/
|   +-- components/
|   |   +-- ClaimsForm.tsx
|   |   +-- DamageAssessment.tsx
|   |   +-- PolicyLookup.tsx
|   |   +-- Dashboard.tsx
|   +-- lib/
|   |   +-- supabase.ts
|   +-- App.tsx
+-- supabase/
|   +-- migrations/
|   |   +-- 001_auth.sql
|   |   +-- 002_claims.sql
+-- tests/
|   +-- claims.test.ts
+-- Dockerfile
+-- docker-compose.yml
+-- README.md
+-- SETUP.md
+-- package.json
```

15. Step 13 - Delivery Tracking (SWON / WON / Tasks)

Who: Manager / Middleware / Leader

When: Post-completion, during delivery phase

Once the AI has generated the product, the human delivery apparatus takes over for tracking, billing, and handoff.

15.1 - SWON (Service Work Order Number)

Purpose: Top-level financial tracking unit. One SWON per approved demand.

API: POST /api/swon (manager or higher_manager only)

Model: SwonRecord in backend/app/db/models.py

Field	Example
`public_id`	`SWON-3A7F2B1C`
`demand_id`	Links to `DMD-3A7F2B1C90`
`customer_loa_ref`	`LOA-2026-INS-0042`
`sow_summary`	"Motor Insurance Claims Portal Delivery"
`lifecycle_state`	Initiated -> Active -> Closed
`total_value_inr`	`INR 75,00,000`

State transitions: Manager moves SWON through Initiated -> Active -> Closed as the engagement progresses. Each transition is audit-logged.

15.2 - WON (Work Order Number)

Purpose: Per-resource billing allocation within a SWON.

API: POST /api/won (manager or higher_manager only)

Model: WonRecord in backend/app/db/models.py

Field	Example
`public_id`	`WON-5B8C3D2E`
`swon_id`	Links to parent SWON
`resource_id`	Team member assigned to this WON
`billable`	`true`
`cost_centre`	`CC-INSURANCE-DEV`
`allocation_pct`	`100` (full-time) or `50` (half-time)
`monthly_value_inr`	`INR 2,50,000`
`state`	Active -> Completed

Multiple WONs can exist under a single SWON (one per allocated resource).

15.3 - Tasks (Granular Work Items)

Purpose: Day-to-day work tracking for the delivery team.

API: POST /api/tasks (manager, leader, or middleware)

Model: Task in backend/app/db/models.py

Field	Example
`public_id`	`TSK-7D9E4F6A`
`demand_id`	Links to the parent demand
`swon_id`	Optionally links to the SWON
`parent_task_id`	Supports hierarchical task trees
`title`	"Integrate damage assessment ML model"
`owner_id`	Assigned team member
`status`	Todo -> In Progress -> Done / Delayed / Reassigned
`est_hours`	`40`
`actual_hours`	`35`
`sla_due_at`	`2026-04-15T17:00:00Z`
`blocked_reason`	`null` or "Waiting for API credentials"

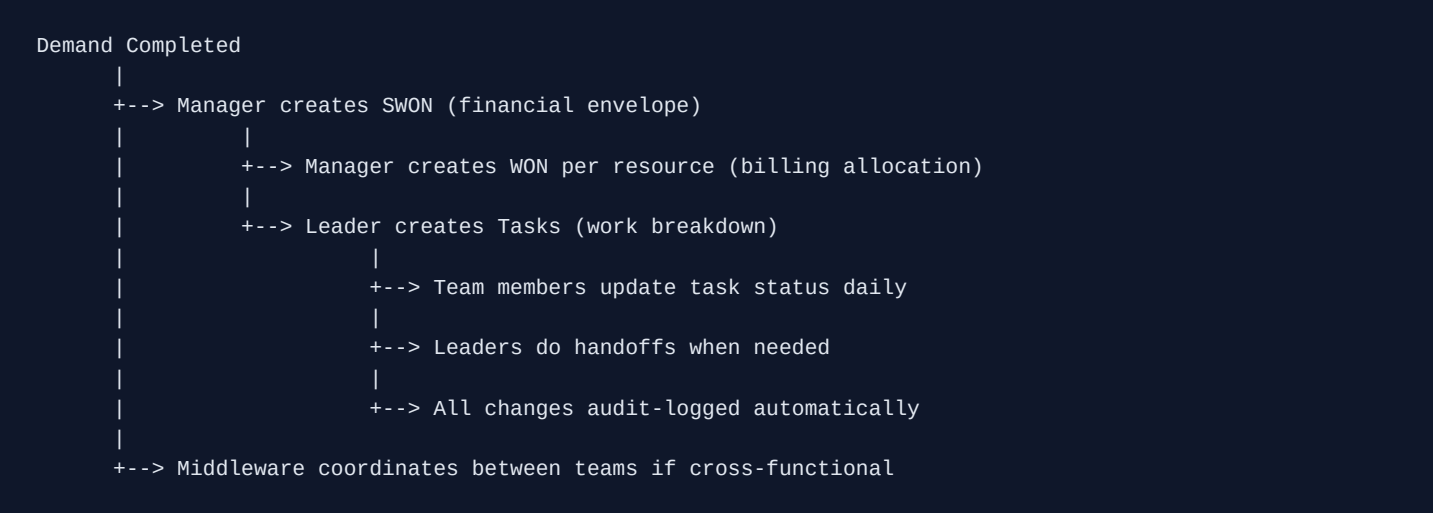
Task Lifecycle



Task Operations

Operation	API	Who Can Do It
Create task	`POST /api/tasks`	Manager, Leader, Middleware
Update status	`PATCH /api/tasks/{id}/status`	Owner, Leader, Manager
Add update/note	`POST /api/tasks/{id}/updates`	Any team member
Handoff/reassign	`POST /api/tasks/{id}/handoff`	Leader, Manager
View timeline	`GET /api/tasks/{id}/timeline`	All team members

Delivery Tracking Flow



16. Step 14 - Dashboards & Reporting

Who: All roles (role-specific views)

Every role has a tailored dashboard pulling from the same underlying data.

Dashboard Summary

Dashboard	Role	Key Metrics
Executive	Executive	Org-wide KPIs, demand trends, delivery rate, SLA b
Higher Manager	Higher Manager	Sanitized portfolio view (no failures, no sensitiv
Manager	Manager	Pending approvals, SLA breaches, blocked tasks, wo
Middleware	Middleware	Cross-team coordination, handoff tracking
Leader	Leader	Task board, member progress, work distribution
Member	Delivery Team	Assigned tasks, personal progress, timeline

Higher Manager Sanitization

The Higher Manager view is intentionally sanitized - it hides:

- * Demands in failed or cancelled stages
- * Raw cost figures and internal estimates
- * Risk factors and blocked-task details
- * Individual team member performance issues

This provides an executive-friendly view focused on positive outcomes and overall portfolio health.

Reports Available

Report	What It Shows	Export Formats
Delivery	Portfolio summary, stage breakdown, completion rates	JSON, CSV, Excel
Team Performance	Tasks per member, completion rates, velocity	JSON, CSV, Excel
Demand Aging	How long demands sit in each stage, age buckets	JSON, CSV, Excel
SLA Compliance	On-time vs overdue tasks, SLA breach analysis	JSON, CSV, Excel
SWON Detail	Financial tracking per SWON with WON breakdown	JSON, CSV, Excel

17. Step 15 - Product Handoff to Client

Who: Manager -> Client

The final step is delivering the product to the client:

What the Client Receives

- * Generated artifacts - Full source code, infrastructure config, tests, documentation
- * Delivery explanation - AI-generated narrative of what was built and why
- * Reuse report - Which components were adapted from past projects
- * Quality report - QA test results and code review findings
- * Setup guide - README.md and SETUP.md with deployment instructions

Client Portal View

The client can access their portal at any time to see:

- * Current demand status and stage
- * Conversation history with the system
- * Final artifacts once completed
- * Delivery explanation and timeline

Post-Delivery

Activity	Who	System Action
Client feedback	Client	Stored in portal messages
Bug fix requests	Client -> Manager	New demand or task update
Change requests	Client -> Manager	New demand linked to original
Project archived	Manager	SWON closed, final audit recorded
Added to reuse library	Manager	Past project created for future reuse

18. Cross-Cutting Concerns

18.1 - Audit Trail

Every single change in the system is tracked via two mechanisms:

Mechanism	What It Catches
Explicit audit records	API-level actions: approvals, stage changes, reass
SQLAlchemy flush hook	All INSERT/UPDATE/DELETE on tracked models (automa

Tracked models: DemandRequest, SwonRecord, WonRecord, Task, TaskHandoff, TeamMember, UserRoleAssignment.

Each audit event captures: tenant_id, entity_kind, entity_id, action, actor_id, diff (column-level), reason (human-provided), timestamp.

18.2 - Multi-Tenancy

Every record in the system carries a tenant_id. All queries are automatically scoped by tenant via scope_filter(), ensuring complete data isolation between organizations.

18.3 - Role-Based Access Control (RBAC)

10 roles with a strict hierarchy:

```
Executive (6)
  +-- Higher Manager (5)
    +-- Manager (4)
      +-- Middleware (3)
        +-- Leader (3)
          +-- Delivery Team (2)
            +-- Member (2)
              +-- Contributor (1)
                +-- Viewer (0)

Client (separate path)
```

Each API endpoint enforces role requirements via require_role(). Higher-hierarchy roles can access lower-level endpoints. The Client role is isolated to the portal path.

18.4 - Real-Time Events

```
Backend Worker --> Redis Pub/Sub --> WebSocket Gateway --> Browser
(forgeos:events:{tenant_id})
```

12 event types covering the full pipeline lifecycle, enabling live progress tracking in the UI.

19. Role Involvement Matrix

This matrix shows which roles are involved at each step of the pipeline:

Step	Client	Manager	Middleware	Leader	Delivery Team	Higher Mgr	Executive
1. Submit demand	*	*					
2. Ingestion							
3. AI Understand							
4. Reuse Detectio							
5. Execution Dec							
6. Resource Alloc							
7. Plan Review		*					
8. Approval Gate		*					
9. Background E:							
10. Monitoring		o					
11. Explanation							
12. Completion		o					
13. SWON Creat		*				o	
14. WON Creatio		*				o	
15. Task Assignn		o	o	*			
16. Task Executi				o	*		
17. Task Handoff		o		*			
18. Dashboard M		*	*	*	*	*	*
19. Reports		*	*	*		* (clean)	*
20. Product Hanc	*	*					

Legend: * = Primary actor | o = Secondary / oversight | (empty) = System-automated

20. Worked Example - Insurance Claims Portal

To make this concrete, here is the full orchestration for a real demand flowing through the system.

Step 1: Client Submission

Client: Bharat Insurance Ltd.

Demand:

"Build a Multi-Channel Motor Insurance Claims Portal with AI-powered damage assessment from uploaded photos. Must support web, mobile, and call-center channels. Integration with existing policy management system required."

Metadata: Industry: Insurance | Priority: High | Timeline: 1-3 months | Budget: INR 50L-2Cr

Step 2: System Ingests

- * Demand ID:** DMD-8B4E2F7A31
- * Raw text stored, word count: 47, char count: 312
- * Audit event: demand.created

Step 3: AI Understanding

Field	Value
Problem Type	`web_application`
Domain	`insurance`
Complexity	`HIGH` (multi-channel + AI + integration)
Urgency	`HIGH`
Required Skills	`react, react_native, python, computer_vision, pos`
Key Features	Claims submission, damage assessment, policy looku
Estimated Scope	45 days

Step 4: Reuse Detection Fires

Past Project	Score	Reusable Components	Verdict
Healthcare Patient Portal	0.82	Auth module, role dashboards, doc	**REUSE**
Retail Inventory Management	0.71	Real-time tracking, notification fran	**PARTIAL**
Banking Loan Processing	0.68	Approval workflows, compliance au	**PARTIAL**

Reuse score: 0.82 -> System will recommend leveraging existing components.

Step 5: Decision

Field	Value
Execution Mode	`HYBRID` (high complexity but strong reuse candida
Estimated Cost	\$12,500
Estimated Time	30 days
Confidence	78%
Reuse Percentage	35%
Risk Factors	Integration complexity, 3rd-party API dependency

Step 6: Allocation

Team (9 members):

Role	Name	Type	Daily Cost
Tech Lead	Priya Sharma	Human	INR 8,000
Frontend Dev	Forge-React	AI	INR 500
Frontend Dev	Amit Patel	Human	INR 5,000
Backend Dev	Forge-Python	AI	INR 500
Backend Dev	Sneha Reddy	Human	INR 5,500
ML Engineer	Vikram Singh	Human	INR 7,000
DevOps	Forge-DevOps	AI	INR 500
QA	Forge-QA	AI	INR 500
Documentation	Forge-Docs	AI	INR 500

Coverage: 92% | Uncovered: react_native (bench resource available)

Step 7-8: Manager Reviews and Approves

Manager Raj reviews the plan, consults the AI copilot about integration risks, and approves. The demand enters execution.

Step 9: Execution

The AI agents generate 47 files in ~8 minutes:

- * 12 React components (claims form, dashboard, policy lookup, etc.)
- * 5 SQL migrations (users, claims, policies, assessments, audit)
- * 3 API route files
- * Dockerfile + docker-compose.yml
- * Test suite (8 test files)
- * README.md + SETUP.md

Step 10-12: Monitoring -> Explanation -> Completion

No issues detected. Explanation generated. Demand marked completed.

Step 13: Delivery Tracking Begins

- * SWON created:** SWON-8B4E2F7A - "Motor Insurance Claims Portal Delivery" - INR 75,00,000
- * WONS created:** 5 WONS for human team members (billable)
- * Tasks created:** 15 tasks spanning frontend, backend, ML integration, testing, deployment

Step 14: Team Executes Over 30 Days

- * Leader assigns tasks to team members
- * Daily status updates tracked
- * 2 task handoffs (ML model integration reassigned when Vikram was blocked)
- * SLA compliance: 13/15 tasks on-time (87%)
- * All changes captured in audit trail

Step 15: Product Handoff

Client receives:

- * Complete source code
- * Deployment guide
- * QA report
- * AI narrative explaining the delivery
- * Reuse report showing 35% time saved from component reuse

SWON closed. Project added to past project library for future reuse.

Summary: The Complete Journey



Document generated for ForgeOS Demand Delivery Pipeline v1.0

Covers the complete client-to-product orchestration flow